

CS F407 / U407 Artificial Intelligence — Project Guidelines

Project component: 30% of course grade
Proposal (10% / 20 marks) + Final Submission (20% / 40 marks)
Team size: 7–8 students

1. Overview & Learning Objectives

In teams of 7–8, you will design and carry out an original AI project connected to what we cover this semester: **Search & Problem Solving, Knowledge Representation & Reasoning, Agents & Game Theory, and Applications of Deep Learning**. The goal is not to reproduce an existing solution, but to take a real problem you care about and apply course concepts to it thoughtfully — and to be able to explain and defend every part of what you built.

Every project must declare one of three tracks. All three are graded equally rigorously, just on different terms:

- **Research** — investigate a question empirically or comparatively (e.g., compare two algorithms, test a hypothesis, analyze a trade-off) and report findings with evidence.
- **Demo** — build a working system that clearly and robustly illustrates a course concept, with a strong live demonstration.
- **Application** — build something aimed at solving a genuine real-world problem, ideally tested on real data or real users.

A list of example ideas per topic and track is in Appendix A — these are starting points, not a menu you must choose from. You are encouraged to propose your own idea based on your team's interests, as long as it connects clearly to course material and is realistically scopeable in one semester.

2. Key Dates

Milestone	Date
AI Project Guidelines announced	31 Aug 2026
Team formation & topic/track finalized	7 Sept 2026
Proposal submission (20 marks)	2 Oct 2026
Mid-project check-in (advisory, not graded)	TBA
Final report, code & AI-usage disclosure due	TBA
Live demo & individual viva sessions	TBA

3. Teams & Track Selection

- Teams of 7–8, self-formed. Coordinate over Google classroom if you need teammates.
- Each team chooses one topic area (Search, KR&R, Agents/Game Theory, or Deep Learning) — or a project that meaningfully combines two, if justified in the proposal — and declares one track (Research, Demo, or Application).
- The declared track shapes how the final submission is graded (Section 5), so choose it deliberately, not as an afterthought.
- Track and topic cannot be changed after the proposal is approved without IC's sign-off.

4. Proposal Submission — 20 marks (10% of course grade)

Submit a 3–5 page proposal containing:

- Team name, member list, declared topic area & track
- GitHub link of the repository
- Problem statement & motivation — what problem, and why it matters
- The specific course concepts/techniques you plan to use (name them precisely — e.g. A*, CSP with arc consistency, minimax with alpha-beta, MDP/value iteration, CNN, Bayesian network)
- Proposed approach & methodology — data/environment, algorithm or architecture, and how you will evaluate success
- Current work done in the area with links and citations.
- Feasibility — a week-by-week milestone timeline and a role assignment covering all 7–8 members
- References, where relevant

Proposal Rubric (20 marks)

Criterion	Marks	What earns full marks
Problem Statement & Motivation	5	Specific, well-motivated problem explicitly tied to a course concept; clear about why it's interesting or worth doing.
Connection to Course Concepts	4	Explicitly names and correctly describes the specific algorithms/techniques from the course that the project will use.
Proposed Approach & Methodology	5	Concrete technical plan — data/environment, algorithm or architecture, and evaluation method — realistic for the team's size and skill level.
Feasibility, Timeline & Roles	3	Realistic week-by-week milestones and a clear division of labor across all 7–8 members, matched to their skills.
Writing Quality & Organization	3	Well-organized, clearly written, properly referenced where relevant.

5. Final Submission — 40 marks (20% of course grade)

Your final submission has four parts:

- **Written report** — Introduction, Background/Related Work, Method, Results/Evaluation, Discussion & Limitations, and a short Individual Contributions section. AI Usage details
- **Code / artifact repository** — with a commit history that reflects work across the semester, not a single last-minute upload (see Section 6).
- **Video + Live demo + individual viva** — demo video + each team runs its system live and answers unscripted questions; every member is individually asked a short question about the project.

Final Submission Rubric (40 marks)

Criterion	Marks	What earns full marks
Technical Depth & Correctness	10	System/experiment runs correctly; reported results match what is demonstrated live; code is reproducible.
Track-Specific Criterion*	8	See track-specific descriptors below.

Criterion	Marks	What earns full marks
Video + Live Demo & Q&A	8	Team runs the system live and answers unscripted questions about design choices and trade-offs convincingly.
Final Report Quality	6	Clear structure; results shown as evidence (plots/tables, not just claims); honest limitations section.
Individual Viva	5	Every member can explain their own contribution and answer a basic question about the system as a whole.
Process & Accountability	3	Evidence of steady work across the semester (commit history/log), honest reflection section.

* *Track-Specific Criterion is judged through the lens of your declared track:*

- **Research** — clear hypothesis, sound experimental design, results analyzed with appropriate rigor, honest discussion of limitations.
- **Demo** — robust and polished; handles edge cases and unexpected input gracefully, not just the ideal path.
- **Application** — solves a real, well-scoped problem with realistic data/users and a clear account of impact and limitations.

6. Use of AI Tools & Academic Integrity

AI tools (Claude, ChatGPT, Copilot, etc.) are allowed and can genuinely help your project — but you own everything you submit. Specifically:

- You may use AI tools for brainstorming, code assistance, writing help, and debugging.
- **You are fully responsible for correctness.** If AI-assisted content is wrong, doesn't run, or contains a fabricated result or citation, it is graded as your own error — "the AI generated it" is not an excuse.
- **Disclose your AI use.** Include a short section in your final report stating which tools you used, for what (e.g. "used an AI assistant to debug an indexing error in Step 3"; "used an AI assistant for wording suggestions on the intro"), and **how you verified the output**.
- **Be able to explain your own work.** In the live viva, every member must be able to explain any part of the project's code, math, or writing they are associated with. If you cannot explain a section you submitted, it will be treated as if you did not do that part.
- **Keep a running trail of progress.** A project assembled entirely in the final 48 hours with no earlier commit history or log will be flagged for review.

7. Team Accountability & Grading

- Individual grade equals the team grade, adjusted if viva performance shows clear evidence of unequal contribution.
- Substantiated free-riding (inability to answer viva questions on one's stated part) results in an individual grade reduction, independent of the team's grade.

Appendix A: Example Project Ideas

These are illustrative starting points, organized by topic area and track — not a required menu. A strong self-proposed idea is always welcome.

Search & Problem Solving

Research

- Compare heuristics for informed search (e.g., 15-puzzle or route-finding) on nodes expanded, time, and memory across problem sizes.
- Compare anytime/real-time search algorithms (RTA*, LRTA*, MCTS variants) on solution-quality vs. time trade-offs.

Demo

- Interactive pathfinding visualizer (A*, Dijkstra, BFS, Greedy Best-First) with live cost/heuristic display and user-placed obstacles.
- CSP solver visualizer for Sudoku/N-Queens/8 puzzle showing backtracking and constraint propagation step-by-step.

Application

- Real campus/city route planner using real map data with multiple objectives (time vs. turns vs. traffic).
- Exam/timetable generator modeled as a CSP, respecting real room, faculty, and student constraints.

Knowledge Representation & Reasoning

Research

- Compare forward- vs backward-chaining inference on a hand-built rule base for efficiency and completeness.
- Compare exact vs. approximate (sampling) inference accuracy/speed trade-offs on a constructed Bayesian network.

Demo

- Small rule-based expert system (e.g., plant-disease ID or device troubleshooting) with an explanation trace of which rules fired.
- Knowledge-graph browser for a domain (courses, movies) supporting natural-language-style queries.

Application

- Degree-requirement advisor: represent your university's requirements as a knowledge base and suggest valid schedules.
- Policy/rule compliance checker for a real regulation set (e.g., hostel rules) using a formal rule base.

Agents & Game Theory

Research

- Compare minimax with alpha-beta vs. Monte Carlo Tree Search across several games, analyzing where each wins and why.
- Simulate different auction/negotiation mechanisms, measuring efficiency, fairness, and strategy-proofness.

Demo

- Simulators for different types of games like Prisoner's Dilemma, Stag Hunt, Bach or Stravinsky with different agent strategies and resulting equilibrium..

- Game-playing agent for a classic board/card game with adjustable AI strength and a visualized search tree.
- Multi-agent pursuit/evasion or predator-prey simulation showing emergent coordination strategies.

Application

- Agent-based traffic-intersection or ride-matching simulator tested on realistic local traffic patterns.
- Resource-allocation tool for a real scenario (e.g., room or task allocation) using mechanism-design principles.

Applications of Deep Learning

Research

- Compare architectures (e.g., CNN vs. Vision Transformer) on a fixed dataset for accuracy/robustness/compute trade-offs.
- Study transfer/few-shot learning effectiveness across domains for a specific task with limited labelled data.

Demo

- Real-time image classifier/detector for a specific domain (local produce, hand gestures, currency notes) with a simple interface.
- Small generative model demo (GAN/diffusion/style-transfer) with a clear before/after showcase.

Application

- Crop/plant disease detection tool for local agriculture using a real image dataset, deployed as a usable app.
- Regional-language NLP tool (e.g., sentiment analysis, transliteration) trained and evaluated on real regional data.

Appendix B: Cross-Area Project Ideas

These ideas deliberately combine two or more of the four topic areas. They tend to be more ambitious, which suits a 7–8 person team well — different sub-teams can own different combined pieces. The area combination for each idea is noted in bold.

Research

- **[Search + Deep Learning]** Learned heuristics for search: train a neural network to estimate heuristic distances for a search problem (e.g. sliding-tile puzzle, robot motion planning) and empirically compare search efficiency and solution quality against classical heuristics.
- **[Knowledge Representation & Reasoning + Deep Learning]** Neuro-symbolic reasoning study: compare a pure neural model against a hybrid neural + knowledge-graph model on a multi-hop reasoning or QA task, measuring accuracy and hallucination rate.
- **[Agents & Game Theory + Deep Learning]** Cooperative vs. competitive multi-agent learning: compare independent-learner vs. centralized-training multi-agent reinforcement learning on a shared game or task, analyzing what strategies emerge.

Demo

- **[Search + Agents & Game Theory + Deep Learning]** AlphaZero-style game agent: Monte Carlo Tree Search guided by a small trained policy/value network for a simplified board game, demoing self-play improvement over training.
- **[Knowledge Representation & Reasoning + Agents & Game Theory]** Argumentation-based negotiation agents: agents that negotiate using an explicit rule/preference base and produce a visualized debate or negotiation transcript explaining each move.
- **[Knowledge Representation & Reasoning + Deep Learning]** Knowledge-grounded QA chatbot: pairs a neural component with a structured knowledge-graph backend, showing the graph reasoning trace alongside each answer for explainability.

Application

- **[Search + Knowledge Representation & Reasoning]** Constraint-aware trip or campus planner: a real itinerary tool that searches over routes and schedules while checking a knowledge base of real constraints (opening hours, accessibility needs, dietary restrictions).
- **[Knowledge Representation & Reasoning + Deep Learning]** Explainable diagnostic assistant: a CNN classifies an image (e.g. crop disease, X-ray finding) while a rule base or knowledge graph cross-checks the result against known symptom-treatment relationships and explains the reasoning in plain language.
- **[Search + Agents & Game Theory + Deep Learning]** Coordinated delivery/shuttle routing system: multiple vehicle-agents coordinate routes via search while a learned model predicts demand and traffic, tested on real local data.